

교통 표지판 이미지 분류에 필요한 프로그램을 개발하는 과정

첨부 자료의 흐름을 따르면, 프로그램은 다음 순서로 개발하면 됩니다.

데이터 확인 → 이미지 분석 → 전처리·데이터 분할 → CNN 구현 → 학습 → 성능 평가 → 성능 개선 → 이미지 예측 → 제출 파일 생성

아래에서는 자료의 내용을 설명하면서, 실제로 실행할 수 있는 TensorFlow 코드와 개발 시 주의할 점을 함께 안내하겠습니다.

1. 먼저, 첨부 파일의 역할부터 확인하기

실제로 첨부된 파일을 확인한 결과는 다음과 같습니다.

첨부 파일	내용	개발 시 활용
1_.pdf	프로젝트 개요와 진행 과정	목표·전체 개발 순서 확인
2_TensorFlow_-_code_.pdf	데이터 구조, 전처리, CNN 학습, 평가, 제출 코드	구현의 기본 참고 자료
submission.csv	Path, label 열로 구성된 제출 양식	최종 예측 결과 저장

중요한 점: 첨부된 submission.csv에는 12,630개의 이미지 경로가 있지만, label 값은 모두 비어 있습니다. 따라서 이 파일은 학습 데이터나 정답 파일이 아니라 **예측 결과를 채워 넣을 제출 양식**입니다.

또한 현재 첨부 파일에는 실제 Train, Test 이미지와 Train.csv, Test.csv가 포함되어 있지 않습니다. 아래 학습 코드를 실행하려면 교육 플랫폼의 데이터 폴더를 사용하거나, 자료에 안내된 원본 데이터를 별도로 준비해야 합니다.

- 자료에 안내된 데이터 출처:
<https://www.kaggle.com/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>
- 교육용 데이터는 원본을 축소한 버전일 수 있으므로, 플랫폼 제출용이라면 플랫폼에서 제공한 데이터와 경로를 우선 사용해야 합니다.

현재 실제 이미지는 없으므로 여기서 학습 정확도를 확인하거나 예측값을 생성할 수는 없습니다.

2. 프로그램이 해야 할 일 정의하기

입력과 출력

프로그램의 기본 기능은 다음과 같습니다.

text복사

```
입력: 교통 표지판 이미지 1장
↓
전처리: RGB 변환 + 32×32 크기 조정
↓
CNN 모델
↓
출력: 표지판 클래스 ID와 예측 확률
```

자료에서 사용하는 GTSRB 데이터는 **0~42의 클래스 ID를 가진 43개 클래스 분류 문제**입니다.

예를 들어 프로그램의 출력은 다음 형태가 됩니다.

python복사

```
{
  "class_id":14,
  "confidence":0.97
}
```

여기서 클래스의 실제 의미를 표시하려면 별도의 **클래스 ID → 표지판 이름** 대응표가 필요합니다. 표지판 이름은 데이터셋 공식 설명을 기준으로 연결해야 합니다.

이번 프로젝트의 범위

이번 프로그램은 **표지판이 찍힌 이미지의 종류를 분류**합니다.

도로 전체 사진에서 표지판 위치를 찾는 것은 별도의 객체 탐지 작업입니다.

text복사

```
이번 프로젝트:
표지판 이미지 → 표지판 종류 분류

후속 확장:
도로 이미지 → 표지판 위치 탐지 → 잘라낸 표지판 분류
```

따라서 이번 CNN만으로 바로 실시간 자율주행 인식 시스템이 완성되는 것은 아닙니다.

3. 개발 환경과 폴더 구성

3.1 권장 환경

처음에는 Jupyter Notebook에서 단계별로 실행하는 것이 좋습니다.

Python 3.10 또는 3.11 환경을 예로 들면 다음과 같이 준비할 수 있습니다.

bash복사

```
python -m venv .venv
```

가상환경 활성화:

bash복사

```
# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate
```

라이브러리 설치:

bash복사

```
python -m pip install --upgrade pip
python -m pip install tensorflow pandas numpy matplotlib pillow scikit-learn jupyter
```

GPU가 없더라도 실행할 수 있지만 학습 시간이 길어질 수 있습니다. GPU 지원 여부는 운영 체제, TensorFlow 버전, 드라이버 환경에 따라 달라집니다.

3.2 폴더 구조

다음 구조를 기준으로 설명하겠습니다.

text복사

```
traffic_sign_project/
├─ data/
│   └─ Train/
│       ├── 0/
│       ├── 1/
│       └─ ...
```

```

|   |   | 42/
|   |   |─ Test/
|   |   |─ Meta/
|   |   |─ Train.csv
|   |   |─ Test.csv
|   |   |─ Meta.csv
|   └─ submission_template.csv
|   └─ outputs/
|   └─ traffic_sign.ipynb

```

첨부된 `submission.csv` 는 원본 보관을 위해 `submission_template.csv` 로 복사해 두면 좋습니다.

교육 플랫폼에서 실행한다면 자료의 경로인 다음 값을 사용할 수 있습니다.

python복사

```
ROOT_DIR= Path("/mnt/elice/dataset")
```

로컬에서 실행한다면 다음처럼 설정합니다.

python복사

```
ROOT_DIR= Path("./data")
```

자료에서 안내한 플랫폼 채점용 파일은 수정하지 않는 것이 좋습니다.

4. 공통 설정 작성하기

아래 코드는 이후 단계에서 공통으로 사용합니다.

python복사

```

import os

# TensorFlow import 전에 설정합니다.
os.environ["TF_CPP_MIN_LOG_LEVEL"]="2"

from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from PIL import Image

```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

SEED=2022
IMG_SIZE=(32,32)
NUM_CLASSES=43
BATCH_SIZE=128

ROOT_DIR= Path("./data")
TEMPLATE_PATH= Path("./submission_template.csv")
OUTPUT_DIR= Path("./outputs")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

tf.keras.utils.set_random_seed(SEED)

# 재현성을 높이기 위한 설정입니다.
# 하드웨어·라이브러리 버전이 다르면 결과가 달라질 수 있습니다.
tf.config.experimental.enable_op_determinism()

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))

```

왜 난수 시드를 고정하나요?

다음 과정에 무작위성이 있기 때문입니다.

- 학습·검증 데이터 분할
- 모델 가중치 초기화
- 학습 데이터 순서 변경
- 이미지 증강

시드를 고정하면 모델 설정을 바꾸었을 때 성능을 비교하기가 쉬워집니다.

5. 데이터와 메타데이터 확인하기

자료에서는 `Train.csv` 와 `Test.csv` 가 이미지 정보를 담고 있다고 설명합니다.

주요 열은 다음과 같습니다.

열	의미
<code>Width</code> , <code>Height</code>	원본 이미지 너비·높이
<code>Roi.X1</code> , <code>Roi.Y1</code>	표지판 영역의 좌상단 좌표

열	의미
Roi.X2 , Roi.Y2	표지판 영역의 우하단 좌표
ClassId	정답 클래스 ID
Path	이미지 상대 경로

5.1 파일 읽기

python복사

```
train_meta= pd.read_csv(ROOT_DIR/"Train.csv")
test_meta= pd.read_csv(ROOT_DIR/"Test.csv")
template= pd.read_csv(TEMPLATE_PATH)

print("학습 메타데이터:", train_meta.shape)
print("테스트 메타데이터:", test_meta.shape)
print("제출 양식:", template.shape)

print(train_meta.head())
print(test_meta.head())
```

5.2 필수 정보 검사

모델을 학습하기 전에 데이터 오류를 먼저 확인해야 합니다.

python복사

```
required_train_columns={"Path", "ClassId"}
required_test_columns={"Path"}

ifnot required_train_columns.issubset(train_meta.columns):
    raise ValueError("Train.csv에 Path와 ClassId가 필요합니다.")

ifnot required_test_columns.issubset(test_meta.columns):
    raise ValueError("Test.csv에 Path가 필요합니다.")

ifnot{"Path", "label"}.issubset(template.columns):
    raise ValueError("제출 양식에 Path와 label이 필요합니다.")

if train_meta[["Path", "ClassId"]].isna().any().any():
    raise ValueError("학습 데이터 경로나 정답에 결측값이 있습니다.")

if test_meta["Path"].isna().any():
    raise ValueError("테스트 데이터 경로에 결측값이 있습니다.")

train_meta["ClassId"]= pd.to_numeric(
    train_meta["ClassId"], errors="raise"
)
```

```

ifnot np.equal(
    train_meta["ClassId"],
    np.floor(train_meta["ClassId"]))
).all():
    raise ValueError("ClassId는 정수여야 합니다.")

train_meta["ClassId"] = train_meta["ClassId"].astype("int32")

ifnot train_meta["ClassId"].between(0,42).all():
    raise ValueError("ClassId는 0~42 범위여야 합니다.")

ifset(train_meta["ClassId"])!=set(range(NUM_CLASSES)):
    raise ValueError("학습 데이터에 43개 클래스가 모두 있는지 확인하세요.")

for name, framein[
    ("Train", train_meta),
    ("Test", test_meta),
    ("제출 양식", template),
]:
    if frame["Path"].duplicated().any():
        raise ValueError(f"{name}에 중복 이미지 경로가 있습니다.")

for name, framein[("Train", train_meta),("Test", test_meta)]:
    missing=[
        pfor pin frame["Path"]
        ifnot(ROOT_DIR/str(p)).is_file()
    ]
    if missing:
        raise FileNotFoundError(
            f"{name} 이미지가 없습니다. 예:{missing[:5]}"
        )

print("기본 데이터 검사 완료")

```

이 검사를 통해 학습 도중 발생할 수 있는 경로 오류를 미리 발견할 수 있습니다.

6. 이미지 데이터 분석하기

자료의 데이터 분석 단계에서는 이미지 크기를 살펴본 후 입력 크기를 **32×32** 로 통일합니다.

6.1 이미지 크기 분포 확인

python복사

```

if{"Width","Height"}.issubset(train_meta.columns):
    print(train_meta[["Width","Height"]].describe())

    train_meta[["Width","Height"]].hist(
        bins=30,
        figsize=(10,4)
    )

```

```
plt.tight_layout()
plt.show()
```

이미지 크기가 서로 다르면 일반적인 배치 학습에서 하나의 텐서로 묶기 어렵기 때문에 일정한 크기로 변환합니다.

이 프로젝트에서는 자료와 동일하게 32×32 를 사용합니다.

6.2 클래스별 데이터 개수 확인

python복사

```
class_counts=(
    train_meta["ClassId"]
    .value_counts()
    .sort_index()
)

print(class_counts)

class_counts.plot(
    kind="bar",
    figsize=(14,4),
    title="Images per Class"
)

plt.xlabel("ClassId")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

이 분석은 클래스 불균형을 확인하기 위한 것입니다.

전체 정확도가 높더라도 데이터가 적은 클래스는 잘 맞이지 못할 수 있으므로, 나중에 클래스별 성능도 확인해야 합니다.

6.3 실제 이미지 확인

python복사

```
samples= train_meta.sample(
    n=min(12,len(train_meta)),
    random_state=SEED
)

plt.figure(figsize=(12,8))

for i,(_, row)inenumerate(samples.iterrows()):
    with Image.open(ROOT_DIR/ row["Path"])as image:
```

```

image= image.convert("RGB")

plt.subplot(3,4, i+1)
plt.imshow(image)
plt.title(f"ClassId={row['ClassId']}")
plt.axis("off")

plt.tight_layout()
plt.show()

```

확인할 사항:

- 이미지가 정상적으로 열리는지
- 표지판이 이미지 안에 있는지
- 흐림, 기울어짐, 밝기 차이가 있는지
- 정답 클래스가 이상해 보이지 않는지

ROI 좌표는 기본 분류 모델에서는 사용하지 않아도 됩니다. 다만 배경을 줄이기 위해 표지판 영역만 잘라 쓰는 실험에는 활용할 수 있습니다. 이때 학습과 실제 추론의 전처리 조건을 일치시켜야 합니다.

7. 학습·검증·테스트 데이터 역할 구분하기

데이터는 다음처럼 사용합니다.

구분	역할
학습 데이터	모델 가중치 학습
검증 데이터	모델 구조·학습률·학습 횟수 선택
테스트 데이터	최종 성능 확인 및 제출 예측

권장 분할은 다음과 같습니다.

text복사

```

Train.csv
├── 80%: 학습
└── 20%: 검증

Test.csv
└── 최종 평가·예측

```

자료에서는 테스트 정확도 90% 이상을 과제 목표로 제시하지만, 모델을 개선할 때마다 테스트 결과를 보고 조정하면 테스트 데이터에도 간접적으로 맞추게 됩니다.

따라서 개선 과정은 검증 데이터로 판단하고, 테스트 데이터는 최종 평가에 사용하는 방식을 권장합니다.

클래스 비율을 유지하며 분할

python복사

```
if class_counts.min()<2:
    raise ValueError(
        "클래스별 이미지가 최소 2장 필요합니다. "
        "데이터를 보충하거나 분할 방식을 조정하세요."
    )

train_df, valid_df= train_test_split(
    train_meta,
    test_size=0.2,
    random_state=SEED,
    stratify=train_meta["ClassId"]
)

train_df= train_df.reset_index(drop=True)
valid_df= valid_df.reset_index(drop=True)

print("학습:",len(train_df))
print("검증:",len(valid_df))
```

`stratify` 는 클래스별 비율을 비슷하게 유지합니다. 축소 데이터가 매우 작으면 검증 데이터 개수가 클래스 수보다 적어 분할이 실패할 수 있으므로 분할 비율을 조정해야 합니다.

또한 동일 표지판을 연속 촬영한 이미지처럼 서로 매우 비슷한 샘플이 있다면, 더 엄격한 평가를 위해 촬영 대상·시퀀스 단위 분할을 검토하는 것이 좋습니다.

8. 제출 양식과 테스트 이미지 순서 맞추기

최종 제출에서 중요한 것은 각 경로에 올바른 예측값을 연결하는 것입니다.

파일 이름을 임의로 정렬해 예측하면 제출 양식과 순서가 달라질 수 있습니다. 따라서 제출 양식의 순서를 기준으로 테스트 데이터를 정렬합니다.

python복사

```
ifset(template["Path"])!=set(test_meta["Path"]):
    raise ValueError(
        "제출 양식과 Test.csv의 이미지 목록이 다릅니다. "
        "올바른 데이터셋을 사용했는지 확인하세요."
    )

test_df= template[["Path"]].merge(
```

```

test_meta,
on="Path",
how="left",
sort=False,
validate="one_to_one"
)

assert test_df["Path"].tolist() == template["Path"].tolist()

```

이렇게 하면 이후 예측 결과를 그대로 제출 양식에 넣을 수 있습니다.

9. 이미지 전처리와 데이터 파이프라인 만들기

자료는 `ImageDataGenerator` 를 사용합니다. 자료를 그대로 재현할 수 있지만, 새 프로그램에서는 `tf.data` 로 구성하면 데이터 읽기와 전처리를 명확하게 관리할 수 있습니다.

아래 방식은 **CSV의 숫자 `ClassId` 를 그대로 사용합니다.**

자료 코드에서 특히 주의할 점

`flow_from_directory()` 는 폴더 이름을 문자열 순서로 처리할 수 있습니다.

text복사

```
0, 1, 10, 11, ..., 2, 20, ...
```

따라서 모델 출력 인덱스가 실제 `ClassId` 와 같다고 가정하면 안 됩니다. 자료가 마지막에 `inverse_label_map` 으로 복원하는 이유가 이것입니다.

아래 코드는 정수 정답을 직접 사용하므로:

text복사

```
모델 출력 인덱스 = 실제 ClassId
```

가 됩니다.

9.1 이미지 읽기 함수

python복사

```

def read_image(path):
    image_bytes = tf.io.read_file(path)

    image = tf.io.decode_image(

```

```

        image_bytes,
        channels=3,
        expand_animations=False
    )

    image.set_shape([None, None, 3])

    image= tf.image.resize(
        image,
        IMG_SIZE,
        method="bilinear"
    )

    # 이 단계에서는 0~255 스케일을 유지합니다.
    # 0~1 정규화는 모델 내부에서 수행합니다.
    return tf.cast(image, tf.float32)

```

처리 과정:

1. 파일 읽기
2. RGB 3채널로 변환
3. **32×32** 로 크기 통일
4. 실수형 텐서로 변환

9.2 데이터셋 생성 함수

python복사

```

def make_dataset(frame, training=False, labeled=True):
    paths=[
        str(ROOT_DIR/ p)
        for pin frame["Path"].astype(str)
    ]

    if labeled:
        labels= frame["ClassId"].to_numpy(dtype=np.int32)

        dataset= tf.data.Dataset.from_tensor_slices(
            (paths, labels)
        )
    else:
        dataset= tf.data.Dataset.from_tensor_slices(paths)

    if training:
        dataset= dataset.shuffle(
            buffer_size=len(frame),
            seed=SEED,
            reshuffle_each_iteration=True
        )

```

```

if labeled:
    dataset= dataset.map(
        lambda path, label:(read_image(path), label),
        num_parallel_calls=tf.data.AUTOTUNE,
        deterministic=True
    )
else:
    dataset= dataset.map(
        read_image,
        num_parallel_calls=tf.data.AUTOTUNE,
        deterministic=True
    )

    dataset= dataset.batch(BATCH_SIZE)
    dataset= dataset.prefetch(tf.data.AUTOTUNE)

return dataset

```

데이터셋 생성:

python복사

```

train_ds= make_dataset(
    train_df,
    training=True,
    labeled=True
)

valid_ds= make_dataset(
    valid_df,
    training=False,
    labeled=True
)

test_x_ds= make_dataset(
    test_df,
    training=False,
    labeled=False
)

```

검증·테스트에서는 순서를 섞지 않습니다.

9.3 데이터 형태 확인

python복사

```

images, labels=next(iter(train_ds))

print("이미지 shape:", images.shape)
print("정답 shape:", labels.shape)

```

```
print("정답 예시:", labels[:10].numpy())
print(
    "픽셀 범위:",
    tf.reduce_min(images).numpy(),
    tf.reduce_max(images).numpy()
)
```

예상 형태:

text복사

```
이미지: (배치 크기, 32, 32, 3)
정답: (배치 크기,)
```

정답이 정수이므로 아래 모델에서는 `sparse_categorical_crossentropy` 를 사용합니다.

10. 자료의 기본 CNN부터 구현하기

자료의 기본 모델은 다음 구조입니다.

text복사

```
이미지
→ Conv2D(filters=5)
→ MaxPooling2D
→ Flatten
→ Dense(43, softmax)
```

모델이 단순하므로 CNN 학습 과정을 이해하기 좋습니다.

python복사

```
defbuild_baseline_model():
    model= tf.keras.Sequential([
        tf.keras.Input(shape=(*IMG_SIZE,3)),

        tf.keras.layers.Rescaling(1.0/255),

        tf.keras.layers.Conv2D(
            filters=5,
            kernel_size=3,
            activation="relu"
        ),

        tf.keras.layers.MaxPooling2D(pool_size=2),

        tf.keras.layers.Flatten(),
```

```

        tf.keras.layers.Dense(
            NUM_CLASSES,
            activation="softmax"
        )
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=1e-3
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model

```

학습:

python복사

```

baseline_model= build_baseline_model()
baseline_model.summary()

baseline_history= baseline_model.fit(
    train_ds,
    validation_data=valid_ds,
    epochs=5
)

baseline_valid_result= baseline_model.evaluate(
    valid_ds,
    return_dict=True
)

print(baseline_valid_result)

```

각 층의 역할

층	역할
Rescaling	픽셀을 0~1 범위로 정규화
Conv2D	표지판의 선, 색상 경계, 형태 등의 특징 추출
MaxPooling2D	특징 맵 크기 축소
Flatten	특징 맵을 1차원 벡터로 변환
Dense(43)	43개 클래스의 예측 확률 출력

자료에는 기본 모델의 학습 정확도 약 90% 이상, 검증 정확도 약 80%, 테스트 정확도 80% 미만이라는 설명이 있습니다. 이는 **자료에 제시된 실행 예시이며, 현재 환경에서 검증된 결과나 항상 재현되는 값은 아닙니다.**

11. 성능 개선 모델 구현하기

자료의 기본 모델은 합성곱층이 하나뿐이라 특징 추출 능력이 제한적일 수 있습니다.

개선 모델에서는 다음을 적용하겠습니다.

- 여러 합성곱층으로 특징 추출 강화
- 작은 회전·이동 증강
- 배치 정규화
- Dropout
- 검증 성능 기준 최적 모델 저장
- 조기 종료와 학습률 감소

11.1 주의: 좌우 반전은 사용하지 않기

교통 표지판에는 방향성이 있는 클래스가 있습니다. 좌우 반전은 표지판의 의미를 바꿀 수 있으므로 무조건 적용하지 않는 것이 좋습니다.

회전도 너무 크게 적용하면 실제 데이터 분포와 달라질 수 있습니다.

11.2 모델 코드

python복사

```
def build_improved_model():
    inputs = tf.keras.Input(
        shape=(IMG_SIZE, 3)
    )

    # 학습과 추론에서 동일하게 정규화합니다.
    x = tf.keras.layers.Rescaling(1.0/255)(inputs)

    # 증강 층은 학습 때만 활성화됩니다.
    x = tf.keras.layers.RandomRotation(
        factor=0.03,
        fill_mode="reflect",
        seed=SEED
    )(x)

    x = tf.keras.layers.RandomTranslation(
        height_factor=0.05,
```

```

        width_factor=0.05,
        fill_mode="reflect",
        seed=SEED+1
    )(x)

    for filters in [32, 64, 128]:
        x = tf.keras.layers.Conv2D(
            filters,
            kernel_size=3,
            padding="same",
            use_bias=False
        )(x)

        x = tf.keras.layers.BatchNormalization()(x)
        x = tf.keras.layers.Activation("relu")(x)

        x = tf.keras.layers.Conv2D(
            filters,
            kernel_size=3,
            padding="same",
            use_bias=False
        )(x)

        x = tf.keras.layers.BatchNormalization()(x)
        x = tf.keras.layers.Activation("relu")(x)

        x = tf.keras.layers.MaxPooling2D(
            pool_size=2
        )(x)

        x = tf.keras.layers.Dropout(0.2)(x)

        x = tf.keras.layers.GlobalAveragePooling2D()(x)

        x = tf.keras.layers.Dense(
            128,
            activation="relu"
        )(x)

        x = tf.keras.layers.Dropout(0.4)(x)

        outputs = tf.keras.layers.Dense(
            NUM_CLASSES,
            activation="softmax"
        )(x)

        model = tf.keras.Model(
            inputs=inputs,
            outputs=outputs
        )

        model.compile(
            optimizer=tf.keras.optimizers.Adam(
                learning_rate=1e-3
            ),
            loss="sparse_categorical_crossentropy",

```

```

        metrics=["accuracy"]
    )

    return model

```

이 모델은 성능 개선을 시도하기 위한 출발점입니다. **90% 이상 정확도를 보장하는 모델은 아니며**, 검증 결과에 따라 조정해야 합니다.

12. 개선 모델 학습하기

python복사

```

best_model_path= OUTPUT_DIR/"best_model.keras"

callbacks=[
    tf.keras.callbacks.ModelCheckpoint(
        filepath=str(best_model_path),
        monitor="val_loss",
        save_best_only=True,
        mode="min"
    ),

    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=8,
        restore_best_weights=True
    ),

    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=3,
        min_lr=1e-6
    )
]

new_model= build_improved_model()
new_model.summary()

history= new_model.fit(
    train_ds,
    validation_data=valid_ds,
    epochs=50,
    callbacks=callbacks
)

```

콜백의 의미

콜백	역할
<code>ModelCheckpoint</code>	검증 손실이 가장 낮은 모델 저장
<code>EarlyStopping</code>	개선이 없으면 학습 중단
<code>ReduceLROnPlateau</code>	성능이 정체되면 학습률 감소

`epochs=50` 은 최대 학습 횟수이며, 조기 종료 조건에 따라 더 일찍 끝날 수 있습니다.
저장된 최적 모델을 다시 읽습니다.

python복사

```
best_model= tf.keras.models.load_model(
    best_model_path
)
```

13. 학습 결과 분석하기

13.1 학습 곡선 확인

python복사

```
defplot_history(history):
    epochs=range(
        1,
        len(history.history["loss"])+1
    )

    fig, axes= plt.subplots(
        1,2,
        figsize=(12,4)
    )

    axes[0].plot(
        epochs,
        history.history["accuracy"],
        label="Train"
    )
    axes[0].plot(
        epochs,
        history.history["val_accuracy"],
        label="Validation"
    )
    axes[0].set_title("Accuracy")
    axes[0].legend()

    axes[1].plot(
        epochs,
```

```

        history.history["loss"],
        label="Train"
    )
    axes[1].plot(
        epochs,
        history.history["val_loss"],
        label="Validation"
    )
    axes[1].set_title("Loss")
    axes[1].legend()

    plt.tight_layout()
    plt.show()

plot_history(history)

```

해석 기준:

현상	가능성	대응
학습·검증 정확도 모두 낮음	과소적합, 학습 부족, 데이터 오류	구조·학습률·정답 확인
학습 정확도는 높지만 검증 정확도는 낮음	과적합 또는 분포 차이	증강·정규화·데이터 보강
검증 손실이 상승하기 시작함	과적합 가능성	최적 시점 모델 사용
손실이 불안정하게 움직임	학습률 과대 등	학습률 감소 검토

증강과 Dropout은 학습 시에만 작동하므로, 검증 정확도가 학습 정확도보다 높게 나타나는 경우도 있습니다.

13.2 클래스별 성능 확인

python복사

```

valid_probabilities= best_model.predict(valid_ds)
valid_predictions= np.argmax(
    valid_probabilities,
    axis=1
)

valid_labels= valid_df["ClassId"].to_numpy()

print(
    "검증 정확도:",
    accuracy_score(valid_labels, valid_predictions)
)

print(

```

```

classification_report(
    valid_labels,
    valid_predictions,
    labels=list(range(NUM_CLASSES)),
    zero_division=0
)
)

```

확인할 지표:

- **Precision:** 해당 클래스라고 예측한 것 중 실제로 맞은 비율
- **Recall:** 실제 해당 클래스 중 찾아낸 비율
- **F1-score:** Precision과 Recall의 균형
- **Macro average:** 클래스별 지표를 동일한 비중으로 평균

13.3 혼동행렬 확인

python복사

```

cm= confusion_matrix(
    valid_labels,
    valid_predictions,
    labels=list(range(NUM_CLASSES))
)

fig, ax= plt.subplots(figsize=(15,15))

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=list(range(NUM_CLASSES))
).plot(
    ax=ax,
    cmap="Blues",
    include_values=False,
    colorbar=False
)

plt.title("Validation Confusion Matrix")
plt.tight_layout()
plt.show()

```

어떤 클래스끼리 자주 혼동되는지 확인한 뒤, 해당 이미지들을 직접 살펴보는 것이 좋습니다.

14. 성능이 부족할 때 개선하는 순서

여러 조건을 동시에 바꾸면 무엇이 효과가 있었는지 알기 어렵습니다. 다음처럼 한 번에 하나씩 바꾸는 것을 권장합니다.

순서	실험	확인할 사항
1	데이터 경로·정답 검사	잘못된 이미지 또는 라벨 여부
2	클래스 ID 대응 검사	모델 인덱스와 실제 ID 불일치 여부
3	합성곱층 추가	특징 추출 능력 개선 여부
4	학습률 조정	안정적 수렴 여부
5	약한 이미지 증강	검증 일반화 개선 여부
6	Dropout 조정	과적합 감소 여부
7	입력을 48×48 , 64×64 로 변경	세부 특징 보존 효과
8	클래스 가중치 적용	희소 클래스 성능 개선 여부

클래스 불균형이 크다면 다음과 같이 가중치를 계산할 수 있습니다.

python복사

```
from sklearn.utils.class_weight import compute_class_weight

classes= np.arange(NUM_CLASSES)

weights= compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=train_df["ClassId"].to_numpy()
)

class_weight={
    int(class_id):float(weight)
    for class_id, weight in zip(classes, weights)
}
```

새 모델의 학습에 적용합니다.

python복사

```
# 별도 실험에서 새 모델을 만든 뒤 실행합니다.
# model.fit(
#     train_ds,
#     validation_data=valid_ds,
#     epochs=50,
#     callbacks=callbacks,
#     class_weight=class_weight
# )
```

클래스 가중치가 전체 정확도를 반드시 높이는 것은 아닙니다. 희소 클래스 Recall과 Macro F1이 어떻게 변하는지도 함께 확인해야 합니다.

15. 테스트 데이터 예측과 최종 평가

모델 선택이 끝났다면 테스트 이미지를 예측합니다.

python복사

```
test_probabilities= best_model.predict(test_x_ds)

test_predictions= np.argmax(
    test_probabilities,
    axis=1
).astype(np.int32)

print("예측 개수:",len(test_predictions))
```

`Test.csv`에 정답 `ClassId`가 있다면 최종 정확도를 계산할 수 있습니다.

python복사

```
if(
    "ClassId" in test_df.columns
    and test_df["ClassId"].notna().all()
):
    test_labels= pd.to_numeric(
        test_df["ClassId"],
        errors="raise"
    ).to_numpy(dtype=np.int32)

    test_accuracy= accuracy_score(
        test_labels,
        test_predictions
    )

    print(f"테스트 정확도:{test_accuracy:.2%}")
else:
    print(
        "테스트 정답이 없으므로 "
        "정확도 계산 없이 제출 파일만 생성합니다."
    )
```

자료의 과제 통과 목표는 테스트 정확도 90% 이상입니다. 다만 이를 달성했는지는 실제 데이터로 평가해야 알 수 있습니다.

16. submission.csv 생성하기

첨부 양식과 동일하게 다음 두 열만 저장합니다.

text복사

```
Path,label
Test/00000.png,예측 클래스
Test/00001.png,예측 클래스
...
```

코드:

python복사

```
submission= template[["Path"]].copy()
submission["label"]= test_predictions

# 제출 전 무결성 검사
assertlen(submission)==len(template)
assert submission["Path"].tolist()== template["Path"].tolist()
assert submission["label"].notna().all()
assert submission["label"].between(0,42).all()

submission_path= OUTPUT_DIR/"submission.csv"

submission.to_csv(
    submission_path,
    index=False
)

print("저장 완료:", submission_path)
print(submission.head())
```

저장 후 다시 읽어 확인합니다.

python복사

```
saved_submission= pd.read_csv(submission_path)

assert saved_submission.columns.tolist()==["Path","label"]
assertlen(saved_submission)==len(template)
assert saved_submission["label"].notna().all()

print("제출 파일 검사 완료")
```

플랫폼이 작업 폴더의 `submission.csv` 를 요구한다면 저장 경로를 다음처럼 바꾸면 됩니다.

python복사

```
submission.to_csv("submission.csv", index=False)
```

핵심은 파일 이름, 열 이름, 이미지 경로 순서, 클래스 ID를 정확히 맞추는 것입니다.

17. 새로운 이미지 1장을 분류하는 기능 만들기

제출 파일 생성과 별개로, 실제 프로그램에서 사용할 단일 이미지 예측 함수를 작성합니다.

python복사

```
def predict_sign(image_path, model, top_k=3):
    image = read_image(
        tf.constant(str(image_path))
    )

    image_batch = tf.expand_dims(
        image,
        axis=0
    )

    probabilities = model.predict(
        image_batch,
        verbose=0
    )[0]

    top_indices = np.argsort(
        probabilities
    )[-top_k:][::-1]

    return [
        {
            "class_id": int(class_id),
            "confidence": float(
                probabilities[class_id]
            )
        }
        for class_id in top_indices
    ]
```

사용 예:

python복사

```
model = tf.keras.models.load_model(
    OUTPUT_DIR/"best_model.keras"
)
```

```

results= predict_sign(
    "./example_sign.png",
    model,
    top_k=3
)

for result in results:
    print(
        f"ClassId={result['class_id']}, "
        f"예측 확률={result['confidence']:.2%}"
    )

```

주의할 점

1. 모델 내부에서 정규화하므로 입력 이미지에 별도로 `/255` 를 적용하지 않습니다.
2. 예측 확률이 높다고 반드시 정답인 것은 아닙니다.
3. 모델은 학습한 43개 클래스 밖의 이미지를 자동으로 구별하도록 학습되지 않았습니다.
4. 일반 도로 사진을 넣으면 표지판 외 배경 때문에 오분류할 수 있습니다.

운영 단계에서는 검증 데이터를 바탕으로 낮은 확률의 예측을 보류하는 정책을 설계할 수 있습니다. 다만 최대 예측 확률만으로 낮은 이미지를 확실하게 판별할 수 있는 것은 아닙니다.

18. 노트북을 프로그램으로 정리하기

실험이 안정화되면 코드를 다음처럼 나누는 것이 좋습니다.

text복사

```

traffic_sign_project/
├─ config.py          # 경로, 이미지 크기, 클래스 수
├─ data.py            # CSV 읽기, 데이터 검사, tf.data 구성
├─ model.py           # CNN 구조
├─ train.py           # 학습, 최적 모델 저장
├─ evaluate.py        # 성능 평가, 혼동행렬
├─ predict.py         # 단일 이미지 예측
├─ submit.py          # 제출 파일 생성
├─ data/
└─ outputs/
    ├─ best_model.keras
    └─ submission.csv

```

각 실행 파일의 역할은 명확하게 분리합니다.

text복사

train.py:

데이터 읽기 → 분할 → 학습 → 모델 저장

evaluate.py:

저장된 모델 읽기 → 평가 데이터 예측 → 지표 출력

predict.py:

저장된 모델 읽기 → 새 이미지 1장 분류

submit.py:

저장된 모델 읽기 → 제출 양식 순서로 예측 → CSV 저장

추가로 모델과 함께 다음 정보를 저장하면 재현과 유지보수가 쉬워집니다.

- 이미지 입력 크기
- 클래스 ID와 이름의 대응표
- 학습·검증 분할 시드
- TensorFlow 버전
- 모델 설정
- 검증 성능
- 사용한 데이터 버전

19. 최종 개발 체크리스트

데이터 준비

- ☐ 실제 **Train**, **Test** 이미지가 있다.
- ☐ **Train.csv**, **Test.csv**의 경로가 실제 파일과 일치한다.
- ☐ 클래스 ID가 0~42 범위이다.
- ☐ 학습 데이터에 43개 클래스가 포함되어 있다.
- ☐ 제출 양식과 테스트 이미지 목록이 일치한다.

전처리·학습

- ☐ 모든 이미지를 RGB로 읽는다.
- ☐ 입력 크기를 동일하게 맞춘다.
- ☐ 정규화가 한 번만 적용된다.

- ☐ 정수 정답에 맞는 손실 함수를 사용한다.
- ☐ 검증·테스트 순서는 섞지 않는다.
- ☐ 검증 성능 기준으로 최적 모델을 저장한다.

평가·제출

- ☐ 전체 정확도뿐 아니라 클래스별 성능도 확인한다.
- ☐ 테스트 정답은 학습에 사용하지 않는다.
- ☐ `label`에 실제 클래스 ID를 저장한다.
- ☐ 출력 열이 `Path`, `label`이다.
- ☐ CSV 저장 시 `index=False`를 사용한다.
- ☐ 저장된 모델로 새 이미지 예측이 가능하다.

권장 개발 순서는 “자료의 기본 모델을 먼저 실행해 전체 파이프라인을 완성한 다음, 개선 모델로 검증 성능을 높이는 것”입니다. 처음부터 복잡한 모델을 만드는 것보다 데이터·클래스 대응·제출 순서가 올바른지 먼저 확인하는 편이 훨씬 중요합니다.

자료 근거: 개요 PDF 23쪽의 ~~프로젝트 목표·진행 순서~~, 코드 PDF 39쪽의 데이터 구조·전처리·기본 CNN, 10~14쪽의 학습·평가·90% 목표·제출 형식.